개인정보보호암호기술

공개키 암호와 키 교환

강원대학교 이세영





이번 시간에 다룰 내용

학습목표

01	일방향 함수와 트랩도어 일방향 함수의 개념을 이해한다
02	RSA의 키 생성 · 암호화 · 복호화 과정을 계산할 수 있다
03	엘가말 암호와 타원곡선 암호의 특징을 구분할 수 있다
04	세 가지 공개키 암호를 같은 기준으로 비교할 수 있다
05	키 교환의 개념과 디피-헬만 방식의 원리 · 한계를 설명할 수 있다

Contents

01

공개키 암호의 원리

대칭키 암호와의 비교, 하이브리드 암호, 일방향 함수, 트랩도어

02

RSA 암호

소인수분해 문제, 키 생성 · 암호화 · 복호화, 지수 연산, 안전성

03

엘가말 암호

이산대수 문제, 엘가말의 구조 · 계산 예 · 특징

04

타원곡선 암호와 비교

타원곡선 암호, 세 가지 비교, 양자 컴퓨터와 미래

05

키 교환과 디피-헬만

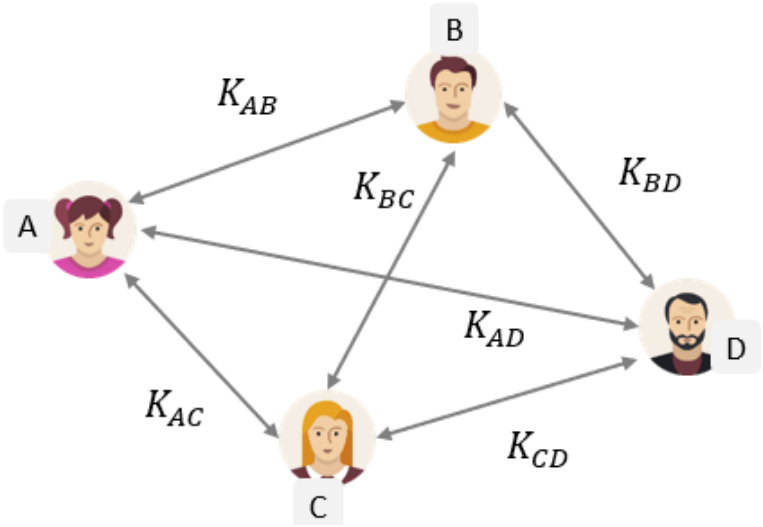
키 교환이란, 디피-헬만의 원리와 계산, 중간자 공격과 대응

대칭키 암호 시스템의 문제점

- 1. 안전한 채널을 통해 송/수신자 간 서로 동일한 키를 사전에 공유할 필요가 있음
  - 키 분배 문제 (Key distribution problem)

- 2. n명이 서로 비밀통신을 하기 위해서는 $\frac{n(n-1)}{2}$개의 키가 필요함
  - Ex) 재학생 1000명이 대칭키 암호 통신하기 위해 필요한 키는 1인당 999개 필요
    - 전체 키의 수: $1000 \times 999 \div 2 = 499{,}500$ 개
  - 키 유지 관리의 어려움

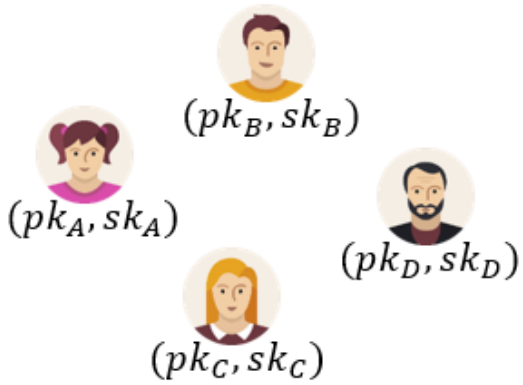
- 3. 송신자나 수신자의 부인방지를 제공하지 못함
  - 부인방지: 메시지를 전송한 송신자나 수신한 수신자가 나중에 그 사실을 부인할 수 없도록 하는 기능

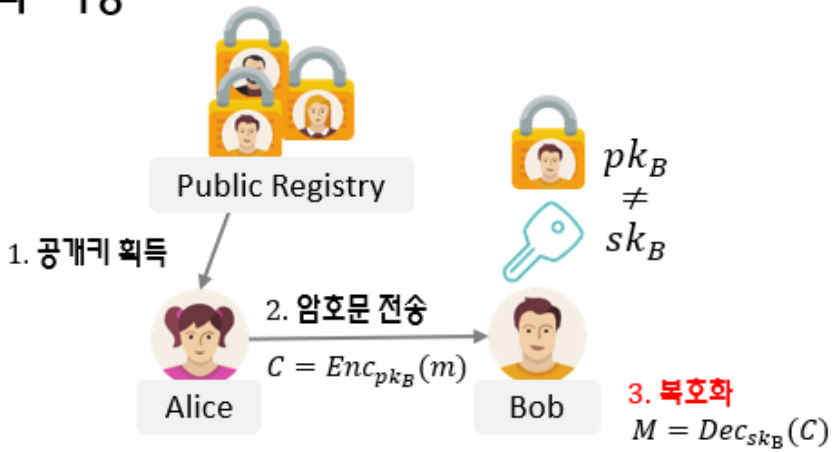


키 분배 문제 · 키 개수 문제 · 부인방지 불가 → 공개키 암호가 등장한 배경

공개키 암호 시스템

- 각자 한 쌍의 키 (공개키 pk, 개인키 sk) 소유
  - 공개키는 모두에게 공개되고 개인키는 비밀로 보관
  - 공개키 pk와 개인키 sk는 서로 수학적인 관계를 가짐
    - pk로부터 sk를 도출하는 것은 계산적으로 불가능 (Computationally Infeasible)
- 암/복호화 과정이 전혀 다름
  - 비대칭키 (asymmetric) 암호시스템이라 하는 이유
  - 개인키를 소유한 사람이 암호문을 복호화 가능





공개키로 암호화 → 짝이 되는 개인키로만 복호화 / 공개키에서 개인키를 구하는 것은 계산적으로 불가능

대칭키 암호와 공개키 암호의 비교

구분	대칭키 암호	공개키 암호
키 구성	하나의 비밀키를 양쪽이 공유	공개키 + 개인키 한 쌍
비밀키 사전 공유	필요	불필요
보유 키 수 (n명)	n − 1 개	자신의 키 쌍 1개
속도	빠름	느림 (수백 배 이상)
같은 안전성의 키 길이	짧음	훨씬 김
대표 알고리즘	AES, SEED, ARIA	RSA, 엘가말, 타원곡선 암호

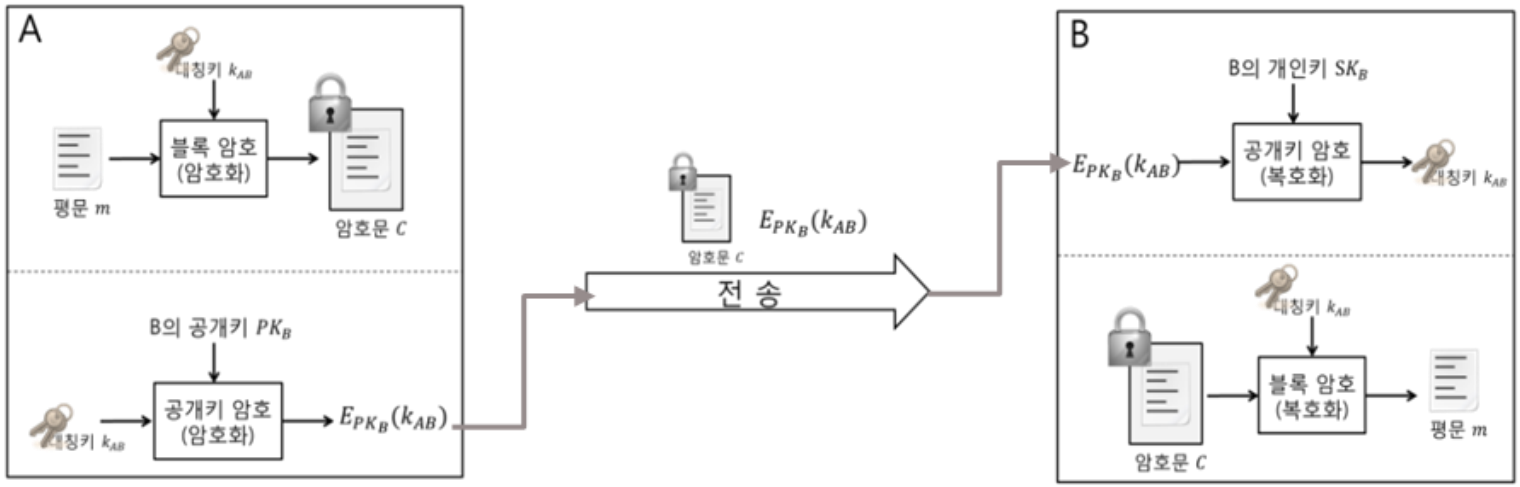
두 방식의 관계

- **공개키 암호는 대칭키 암호를 대체하는 기술이 아님**
- **키를 나눠 갖기 어려운 문제 → 공개키 암호가 해결**
- **속도가 느린 문제 → 대칭키 암호가 해결**

– 서로의 약점을 보완하는 관계 → 실제로는 함께 사용

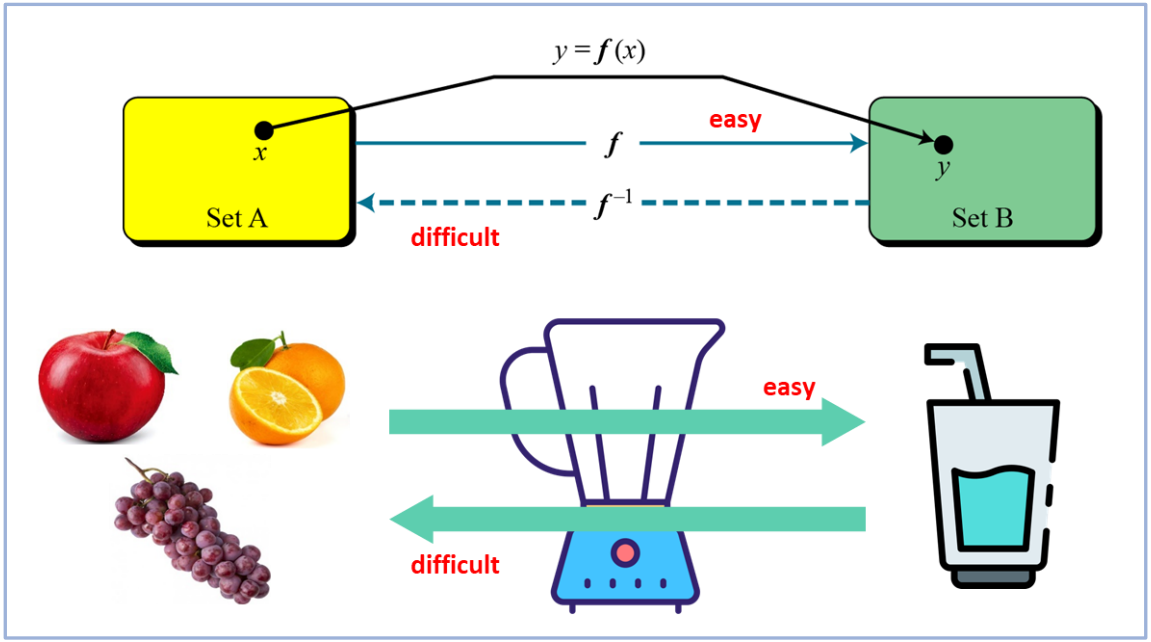
하이브리드 암호 시스템

- 사전 비밀키 공유 (또는 분배) 가 필요한 대칭키 암호시스템의 단점을 보완
- 암/복호화 소요 시간 및 계산량이 많은 공개키 암호시스템의 단점을 보완
- 공개키 암호시스템과 대칭키 암호시스템을 함께 사용
  - 대칭키 암호시스템의 비밀키 k를 공개키 암호시스템으로 암호화하여 분배
  - 분배된 비밀키를 이용하여 대용량의 데이터를 대칭키 암호 시스템으로 암호화



공개키 암호로 평문을 직접 암호화할 수도 있지만, 대용량 데이터에는 하이브리드 구조를 사용

일방향 함수 (One-way Function)



일방향 함수

- **x → f(x) 계산은 쉬움**
- **y가 주어졌을 때 x를 찾는 것은 매우 어려움**

– 과일 → 주스는 쉽지만,
주스 → 과일은 불가능

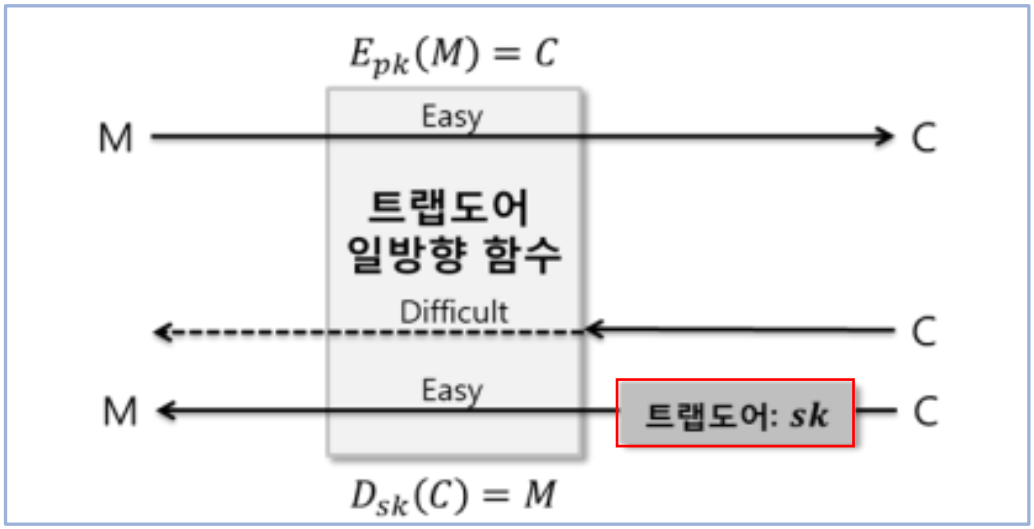
"어렵다"의 의미

- **이론적으로 불가능하다는 뜻이 아님**
- **모든 경우를 시도하면 언젠가는 찾을 수 있음**

– 다만 그 시간이 현실적으로 감당할 수 없음
– = 계산적으로 불가능

일방향 함수만으로는 암호를 만들 수 없다 — 정당한 수신자는 되돌릴 수 있어야 하기 때문

트랩도어 일방향 함수





바닥에 숨겨진 비밀 문 (trapdoor)

트랩도어 일방향 함수

- 일반적으로는 되돌리기 어려움

– 비밀 정보(트랩도어)를 알면 쉽게 되돌릴 수 있음

- 그 비밀 정보가 바로 개인키

남은 질문

- 어떤 수학 문제가 이 조건을 만족하는가?

– 한쪽 방향은 쉽고, 반대는 어렵고,
– 비밀 정보를 알면 반대도 쉬운 문제

암호마다 사용하는 문제가 다름 : RSA → 소인수분해 문제 / 엘가말 · 타원곡선 암호 → 이산대수 문제

RSA 암호의 개요

RSA 암호

- 1978년 발표 (MIT)
- Rivest · Shamir · Adleman 세 사람의 성 첫 글자
- 소인수분해 문제의 어려움에 기반

– 가장 널리 쓰이는 공개키 암호

사용되는 곳

- 웹 통신에서 서버의 신원 확인 (인증서)
- 전자문서의 서명
- 대칭키를 안전하게 전달

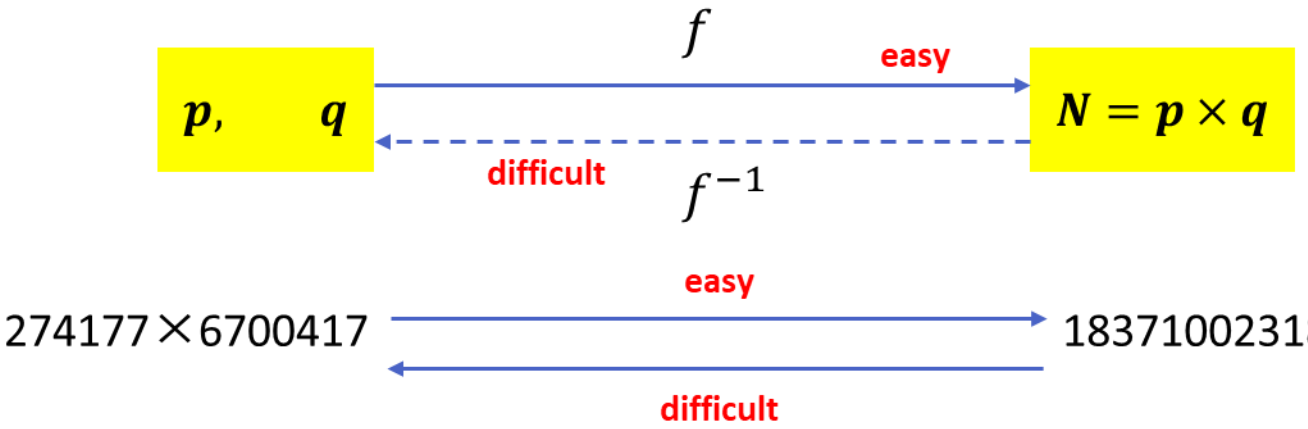
기호	의미	공개 여부
p, q	서로 다른 큰 소수	비밀
n = p × q	모듈러스	공개
e	공개키를 이루는 수 (n과 함께 공개)	공개
d	개인키가 되는 수	비밀

공개키 = (n, e)
개인키 = d

- **p와 q는 키 생성 후 비밀 보관 또는 폐기 (알려지면 개인키가 계산됨)**

소인수분해의 어려움 위에 세워진 가장 대표적인 공개키 암호

RSA가 사용하는 문제 : 소인수분해



왜 공개키 암호의 바탕이 되는가

- 두 소수 p, q를 아는 사람 → N을 쉽게 계산
- N만 아는 사람 → p, q를 알 수 없음

– p와 q = 개인키를 만드는 재료, N = 공개키의 일부

- 공개키를 알려 주어도 개인키가 안전한 이유

곱셈은 쉽고 소인수분해는 어렵다 — 이 비대칭이 공개키 암호를 만든다

소인수분해는 실제로 얼마나 어려운가

소인수분해 기록	자릿수	비트 수	비고
2009년	232자리	768비트	수백 대의 컴퓨터로 2년
2020년 2월	250자리	829비트	여러 연구 기관이 공동 수행
2026년 9월	260자리	**862비트**	현재까지의 기록

자릿수가 조금만 늘어도

- **소요 시간은 자릿수에 비례하지 않고 훨씬 가파르게 증가**
- **232자리 → 250자리에도 다시 몇 년의 연구와 막대한 자원이 필요**

오늘날 사용하는 2048비트는?

- **2048비트 = 10진수로 약 617자리**

– 기록으로 깨진 260자리와 비교할 수 없을 만큼 큼

키를 늘리는 비용은 약간의 계산 시간이지만, 공격 비용은 상상할 수 없을 만큼 커진다
→ 방어하는 쪽에 유리한 비대칭

RSA 키 생성 절차

1	서로 다른 큰 소수 p, q 선택	실제로는 각각 1024비트 정도
2	$n = p \times q$ 계산	공개키의 일부가 됨
3	$L = (p-1) \times (q-1)$ 계산	p, q를 알아야 계산할 수 있는 값
4	$1 < e < L$ 이고 $\gcd(e, L) = 1$ 인 e 선택	공개키를 이루는 수
5	$e \times d \equiv 1 \pmod{L}$ 인 d 계산	L에 대한 e의 곱셈 역원

d를 구하려면 L이 필요하고, L을 구하려면 p와 q가 필요하다
→ n에서 p, q를 찾을 수 없다면 d도 알 수 없다 (RSA 안전성의 핵심)

RSA 키 생성 계산 예

단계	계산	결과
① 소수 선택	p, q 선택	p = 5, q = 11
② n 계산	n = p × q	n = 5 × 11 = 55
③ L 계산	L = (p−1)(q−1)	L = 4 × 10 = 40
④ e 선택	1 < e < 40, gcd(e, 40) = 1	**e = 3**
⑤ d 계산	e × d ≡ 1 (mod 40)	**d = 27 (3 × 27 = 81 = 2×40 + 1)**

공개키 (n, e) = (55, 3) **개인키 d = 27**

작은 수를 쓰면 안 되는 이유

- **n = 55 → 5 × 11 임을 금방 알 수 있음**

– L = 40, d = 27 까지 바로 계산됨

- **실제로는 n을 2048비트 이상으로 사용**

RSA의 큰 소수는 어떻게 찾는가

① 난수 생성	1024비트 크기의 홀수를 무작위로 하나 뽑음
② 소수 판정	그 수가 소수인지 검사
③ 반복	소수가 아니면 버리고 다시 ①로

소수가 나올 때까지 반복

① 소수는 충분히 자주 나온다

- **1024비트 근처에서는 약 710개에 하나꼴로 소수**

– 홀수만 시도하면 평균 350번 정도면 하나 발견

- **컴퓨터에게는 매우 짧은 시간**

② 소수 판정은 빠르게 할 수 있다

- **작은 수로 하나씩 나누어 보는 것은 불가능**
- **확률적 소수 판정법 사용**

– 소수가 아니면 높은 확률로 걸러내는 검사를 반복

– 충분히 반복하면 틀릴 확률이 사실상 0

주의 사항

- **소수를 뽑는 난수는 반드시 예측 불가능해야 함**

– 예측 가능하면 공격자도 같은 소수를 찾음

- **두 소수가 서로 너무 가까우면 안 됨**

큰 소수 찾기 = 무작위로 뽑고 검사하기를 반복하는 일

RSA 암호화와 복호화

RSA 암호화와 복호화

암호화 : c = m^e mod n　　복호화 : m = c^d mod n

(단, 평문 m < n)

계산 예 : 공개키 (55, 3), 개인키 27, 평문 m = 9

과정	계산	결과
암호화	9^3 mod 55 = 729 mod 55	c = 14
복호화	14^{27} mod 55	m = 9

확인할 점

- 두 식의 모양은 같고 지수만 다름 (e ↔ d)
- 평문은 반드시 n보다 작아야 함

– 긴 메시지는 블록으로 나누거나 하이브리드 방식 사용

- 큰 거듭제곱도 곱할 때마다 나머지를 취하면 작은 수만 다루게 됨

지수 연산을 효율적으로 하는 방법

두 가지 요령

- **① 곱할 때마다 모듈러스로 나눈 나머지를 취한다**

– 중간에 나머지를 취해도 결과는 같음 → 수가 작게 유지됨

- **② 제곱을 반복해 필요한 것만 골라 곱한다 (제곱-곱 방법)**

– 지수를 2의 거듭제곱의 합으로 쪼갬 : 27 = 16 + 8 + 2 + 1

① 제곱을 반복해 미리 구하기 ($14^x \bmod 55$)

거듭제곱	계산	mod 55
14^2	14 × 14 = 196	31
14^4	31 × 31 = 961	26
14^8	26 × 26 = 676	16
14^{16}	16 × 16 = 256	36

② 필요한 것만 곱하기 ($14^{27} = 14^{16} \times 14^8 \times 14^2 \times 14^1$)

곱하기	계산	mod 55
$14^{16} \times 14^8$	36 × 16 = 576	26
$\times 14^2$	26 × 31 = 806	36
$\times 14^1$	36 × 14 = 504	**9**

지수	그냥 곱하면	제곱-곱 방법
27	26 번	**7 번**
2048비트 지수	약 2^{2048} 번 (불가능)	**약 3,000 번**

RSA의 전체 동작

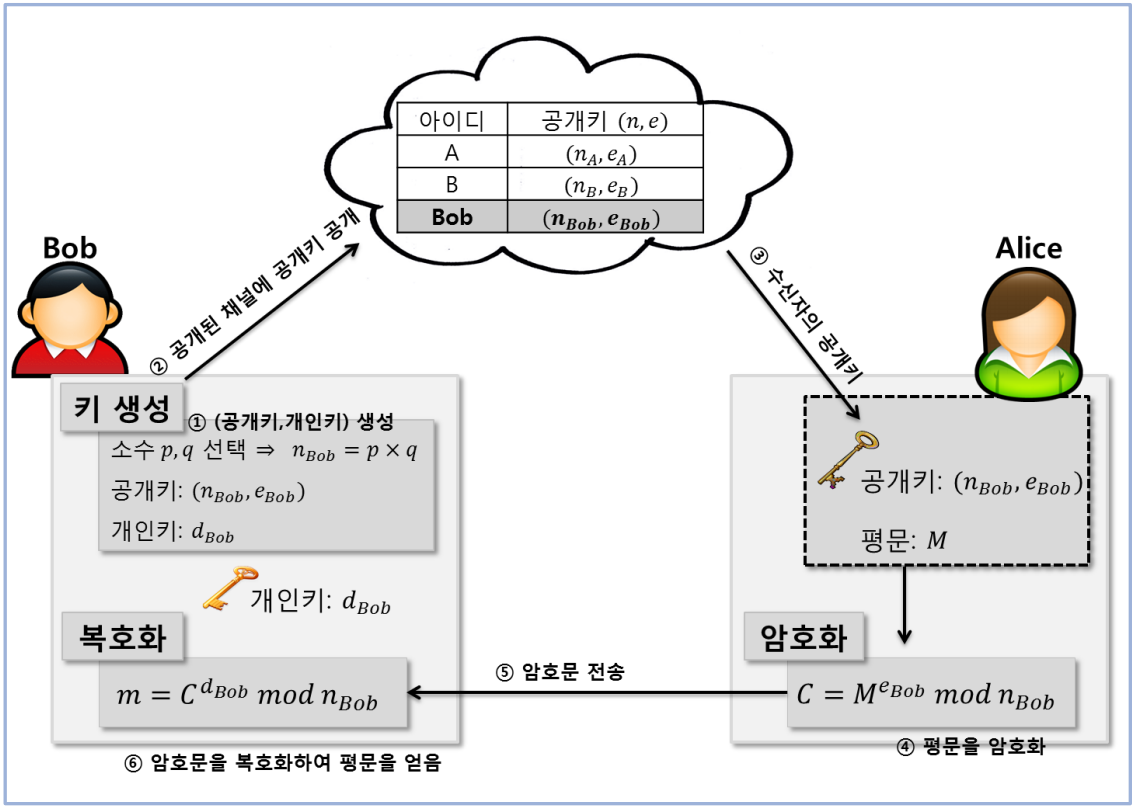
순서

- ① 밥이 키 쌍 생성
- ② 공개키를 공개 저장소에 등록
- ③ 앨리스가 밥의 공개키 획득
- ④ 공개키로 암호화해 전송
- ⑤ 밥이 개인키로 복호화

각 단계에서 누가 무엇을 아는가

- 밥 : p, q, n, e, d 전부
- 앨리스 : 밥의 공개키 (n, e)
- 공격자 : 공개키 (n, e)와 암호문 c

– 공격자가 평문을 알려면
n을 소인수분해해야 함



RSA의 안전성

RSA 문제

- **공개키 (n, e)와 암호문 c가 주어졌을 때 평문 m을 구하는 문제**

– 알려진 유일한 방법 : n을 소인수분해 → p, q → d 계산

안전성의 근거

- **n을 소인수분해할 수 있으면 RSA는 깨짐**
- **소인수분해보다 쉬운 공격은 아직 발견되지 않음**

– 증명된 것이 아니라 "아직 발견되지 않은" 것

지켜야 할 사항

- **n은 2048비트 이상**
- **두 소수가 서로 너무 가깝지 않아야 함**

– p, q가 비슷하면 n의 제곱근 근처에서 찾을 수 있음

- **소수를 고를 때 안전한 난수를 사용**

– 예측 가능하게 고르면 공격자도 같은 소수를 찾음

같은 평문 + 같은 공개키 → 항상 같은 암호문
→ 실제로는 평문에 난수를 섞는 패딩을 적용해 매번 다른 암호문이 나오게 함

RSA 사용 시 주의 사항

① 키 길이	2048비트 이상 사용 (1024비트는 권장되지 않음)
② 패딩	평문을 그대로 거듭제곱하지 않고 난수를 섞는 표준 패딩 적용
③ 용도 분리	하나의 키 쌍을 암호화와 전자서명에 함께 사용하지 않음
④ 개인키 관리	안전한 저장소에 보관하고 외부로 내보내지 않음
⑤ 구현	연산 시간 · 소비 전력 차이로 개인키가 드러나지 않도록 구현

패딩 없이 쓰는 RSA는 안전하지 않다 — 알고리즘만이 아니라 사용 방법까지 표준을 따라야 함

또 하나의 어려운 문제 : 이산대수 문제

- $7^0 \bmod 13 = 1$
- $7^1 \bmod 13 = 7$
- $7^2 \bmod 13 = 10$
- $7^3 \bmod 13 = 5$
- **$7^4 \bmod 13 = 9$**
- $7^5 \bmod 13 = 11$
- $7^6 \bmod 13 = 12$
- $7^7 \bmod 13 = 6$
- $7^8 \bmod 13 = 3$
- $7^9 \bmod 13 = 8$

$7^x \bmod 13$ 의 값 (x = 1 ~ 9)

왜 어려운가

- **보통의 대수 : $7^x = 49 \rightarrow x = 2$ (쉽게 구함)**
- **모듈러 연산 : 결과가 규칙 없이 흩어짐**

– 결과만 보고 지수를 짐작할 수 없음

- **모듈러스가 수백 자리가 되면**

– 표를 만들 수 없고 효율적인 방법도 알려지지 않음

이산대수 문제 (Discrete Logarithm Problem)

- **$\alpha^x \bmod p$ 를 계산하는 것은 쉬움**
- **$\alpha^x \bmod p = \beta$ 를 만족하는 x를 찾는 것은 어려움**

어려운 문제	한 방향은 쉬움	반대 방향은 어려움	이 문제에 기반한 기술
소인수분해	$p \times q = N$	$N \rightarrow p, q$	RSA
이산대수	$\alpha^x \bmod p$ 계산	결과에서 x 찾기	엘가말, 타원곡선 암호, 디피-헬만

엘가말 암호 (ElGamal) 의 구조

앨리스 (보내는 사람)

밥 (받는 사람)

① 비밀값 d 선택
② β = α^d mod p 계산

→ 공개키 (p, α, β), 개인키 d

공개키 (p, α, β) 공개

③ 임시 비밀값 i 선택
④ 임시 공개값 c_1 = α^i mod p
⑤ 마스킹 키 = β^i mod p
⑥ 마스킹된 평문 c_2 = 평문 × 마스킹 키 mod p

암호문 (c_1, c_2) 전송

⑦ 마스킹 키 = c_1^d mod p (앨리스와 같은 값)
⑧ 평문 = c_2 ÷ 마스킹 키 mod p

왜 같은 마스킹 키가 나오는가

- 앨리스 : $\beta^i = (\alpha^d)^i$
- 밥 : $c1^d = (\alpha^i)^d$

– 둘 다 $\alpha^{d \times i}$ mod p → 같은 값

밥의 공개키와 앨리스의 임시 비밀값으로 "둘만 아는 일회용 키"를 만들어 평문을 가린다

엘가말 암호의 계산 예와 특징

단계	계산	결과
밥의 키	$d = 12,\ \beta = 2^{12} \bmod 29$	$\beta = 7$
앨리스의 임시값	$i = 5,\ c_1 = 2^5 \bmod 29$	$c_1 = 3$
마스킹 키	$\beta^i = 7^5 \bmod 29$	16
암호화	$c_2 = 15 \times 16 = 240 \bmod 29$	**암호문 (3, 8)**
복호화	$3^{12} \bmod 29 = 16,\ 8 \div 16 \bmod 29$	**평문 15**

※ 모듈러 연산에서 나누기 = 곱셈 역원을 곱하기 (16의 역원은 20, $8 \times 20 = 160 \equiv 15 \bmod 29$)

특징

- **매번 다른 임시값 → 같은 평문도 매번 다른 암호문**
  - RSA는 패딩이 없으면 같은 암호문
- **암호문이 두 값 → 평문의 두 배 길이**
- **이산대수 문제에 기반**

주의

- **임시 비밀값 재사용 금지**
  - 같은 값을 두 번 쓰면 두 암호문의 관계에서 평문이 드러남

엘가말 = 일회용 마스킹 키로 평문을 가리는 공개키 암호

엘가말 암호의 특징

① 매번 다른 암호문

- **보낼 때마다 새로운 임시 비밀값 사용**

– 후보 평문을 암호화해 비교하는 공격이 통하지 않음

② 암호문이 두 배 길이

- **암호문 = (임시 공개값, 마스킹된 평문)**

– 저장 공간과 통신량이 두 배로 늘어남

③ 임시 비밀값 재사용 금지

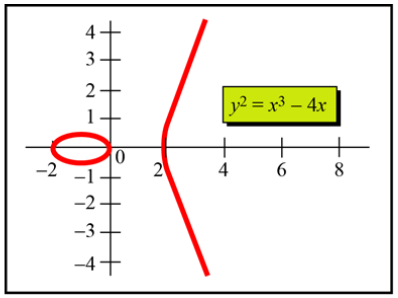
- **같은 임시값을 두 번 쓰면 마스킹 키도 같아짐**

– 두 암호문의 관계에서 평문 정보가 드러남

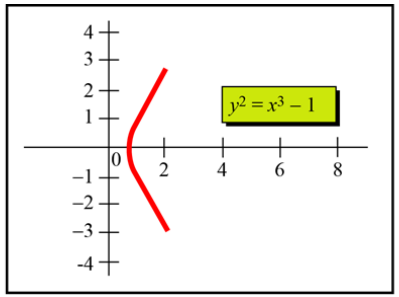
- **"같은 값을 두 번 쓰지 않는다"는 원칙이 그대로 적용**

구분	RSA	엘가말
기반 문제	소인수분해	이산대수
같은 평문의 암호문	항상 같음 (패딩 필요)	매번 다름
암호문 길이	평문과 비슷	평문의 두 배
암호화 때마다 난수	패딩에 사용	임시 비밀값으로 반드시 필요

타원곡선 암호 (ECC)



→ 3 개의 실근 (x = −2, x = 0, and x = 2)



→ 오직 1개의 실근 (x = 1), 두개의 허근

타원곡선의 예 : $y^2 = x^3 + ax + b$

기본 아이디어

- 곡선 위의 점들 사이에 덧셈 연산을 정의
- 점 P를 k번 더해 점 Q를 만드는 것은 쉬움

– P와 Q만 보고 k를 알아내는 것은 어려움

- k = 개인키, Q = 공개키

안전성 수준	대칭키 암호	RSA	타원곡선 암호
보통	112비트	2048비트	224비트
권장	128비트	3072비트	**256비트**
높음	192비트	7680비트	384비트

같은 안전성에 훨씬 짧은 키 → 저장 공간 · 통신량 · 계산량 감소
스마트카드, 사물인터넷 기기, 모바일 환경에서 널리 사용

세 가지 공개키 암호의 비교

구분	RSA	엘가말	타원곡선 암호
기반 문제	소인수분해	이산대수	타원곡선 이산대수
공개키 구성	(n, e)	(p, α, β)	곡선 위의 점
암호화 방식	평문을 e제곱	일회용 마스킹 키를 곱함	일회용 마스킹 키를 더함
암호문 특징	평문과 비슷한 길이 (패딩 없으면 같은 암호문)	평문의 두 배 길이 매번 다른 암호문	평문의 두 배 길이 매번 다른 암호문
최소 권장 키 길이	2048비트	2048비트	**224비트**

세 방식의 공통점

- **한쪽 방향은 쉽고 반대 방향은 어려운 문제 위에 세워져 있음**
- **모두 느리기 때문에 대용량 데이터를 직접 암호화하지 않음**

– 대칭키를 정하거나 짧은 데이터를 보호하는 데 사용

공개키 암호의 미래 : 양자 컴퓨터

양자 컴퓨터가 등장하면

- **소인수분해와 이산대수를 효율적으로 푸는 알고리즘이 이미 존재**

– RSA · 엘가말 · 타원곡선 암호 모두 안전성을 잃음

- **대칭키 암호는 키 길이를 늘려 대응 가능하지만**

– 공개키 암호는 키를 늘려도 소용없음

대응 : 양자내성암호

- **전혀 다른 수학 문제에 기반한 새로운 공개키 암호**
- **현재 표준화가 진행되어 일부 알고리즘이 채택됨**

– 기존 시스템에서 알고리즘을 교체할 수 있도록 설계해 두는 것이 중요

"지금 저장해 두었다가 나중에 해독한다"

- **공격자가 지금 오가는 암호문을 저장해 둘 수 있음**
- **먼 미래에 양자 컴퓨터로 해독 가능**

– 지금 보호한 정보라도 미래에 읽힐 수 있다는 뜻

- **보관 기간이 긴 정보(의료 · 유전 정보 등)일수록 더 중요**

암호의 안전성은 영원하지 않다 — 알고리즘을 바꿀 수 있도록 미리 준비해 두어야 한다

키 교환 : 암호가 아니라 "키를 만드는" 기술

방법 ① 키 전송 (Key Transport)

- **한쪽이 대칭키를 만들어 상대의 공개키로 암호화해 전송**
- **받는 쪽은 개인키로 복호화해 같은 키 획득**

– RSA 같은 공개키 암호를 사용
– 한쪽이 키를 정함

방법 ② 키 합의 (Key Agreement)

- **양쪽이 각각 비밀값을 정하고 공개 정보를 교환**
- **결과적으로 양쪽이 같은 키를 만들어 냄**

– 어느 한쪽이 정하지 않고 함께 만들어 냄
– 대표 : 디피-헬만 키 교환

구분	키 전송	키 합의
키를 정하는 주체	한쪽	양쪽이 함께
사용하는 기술	RSA 등 공개키 암호를 활용	디피-헬만 등 키 합의 방식

공개키 암호 = 메시지를 암호화하는 기술 / 키 교환 = 같은 키를 만들어 내는 기술

디피-헬만 키 교환의 아이디어

① 공통 색을 공개로 정한다

공통 색

② 각자 비밀 색을 정한다

앨리스의 비밀 색

밥의 비밀 색

③ 공통 색 + 자기 비밀 색을 섞어 전송 (공개되어도 안전)

섞은 색 A → 섞은 색 B

섞은 색 B → 섞은 색 A

④ 받은 색에 다시 자기 비밀 색을 섞는다 → 양쪽이 같은 색

공통 색 + 앨리스 색 + 밥 색 = 공통 색 + 밥 색 + 앨리스 색

공격자는?

- 공통 색과 주고받은 섞인 색은 볼 수 있음
- 그러나 섞인 색에서 비밀 색을 분리할 수 없음

– 색 섞기는 쉽고 분리는 어렵다 = 거듭제곱은 쉽고 이산대수는 어렵다

디피-헬만 키 교환의 절차

공개 정보 : 큰 소수 p, 1 < α < p 인 수 α (공격자도 알고 있음)

앨리스	밥
① 비밀값 a 선택 (혼자만 안다)	① 비밀값 b 선택 (혼자만 안다)
② 공개값 A = α^a mod p 계산	② 공개값 B = α^b mod p 계산
	A 전송 →
B 전송 ←	
③ 공유 비밀 = B^a mod p	③ 공유 비밀 = A^b mod p

앨리스 : (α^b)^a 밥 : (α^a)^b → 둘 다 α^(a×b) mod p = 같은 값
이 값을 대칭키로 사용해 실제 데이터를 암호화

디피-헬만 계산 예

공개 정보 : p = 29, α = 2

단계	앨리스	밥
① 비밀값	a = 5	b = 12
② 공개값 계산	$A = 2^5 \bmod 29 = 32 \bmod 29 = 3$	$B = 2^{12} \bmod 29 = 4096 \bmod 29 = 7$
③ 서로 전송	A = 3 을 밥에게	B = 7 을 앨리스에게
④ 공유 비밀	**$7^5 \bmod 29 = 16$**	**$3^{12} \bmod 29 = 16$**

두 사람은 비밀값을 모른 채 같은 값 16을 얻었다

공격자가 볼 수 있는 것

- **공개 정보 p = 29, α = 2**
- **주고받은 공개값 A = 3, B = 7**

– 공유 비밀 16을 구하려면 $2^x \bmod 29 = 3$ 인 x를 찾아야 함 (= 이산대수 문제)

- **실제로는 2048비트 이상의 소수를 사용**

중간자 공격 : 공격자는 어떻게 키를 만드는가



	앨리스가 계산	공격자가 계산	밥이 계산
앨리스 ↔ 공격자	$(\alpha^m)^a = \alpha^{m\times a}$	$A^m = \alpha^{a\times m}$	—
공격자 ↔ 밥	—	$B^m = \alpha^{b\times m}$	$(\alpha^m)^b = \alpha^{m\times b}$

결과

- 공격자는 두 개의 키를 가짐 : 앨리스와 공유하는 키, 밥과 공유하는 키
- 앨리스의 메시지를 복호화해 읽고, 다시 암호화해 밥에게 전달

– 두 사람은 서로 직접 공유했다고 믿어 아무 이상도 느끼지 못함

이산대수 문제는 전혀 깨지지 않았다 — 멜로리는 a도 b도 알아내지 못했다
공격이 성공한 이유는 "받은 값이 누구에게서 왔는지" 확인하지 않았기 때문

핵심 정리 (1) : 공개키 암호

공개키 암호의 원리	일방향 함수 : 한쪽은 쉽고 반대는 어려움 트랩도어 : 비밀 정보(= 개인키)를 알면 반대 방향도 쉬워짐
RSA (소인수분해)	$n = p \times q$, $L = (p-1)(q-1)$, $\gcd(e, L) = 1$, $e \times d \equiv 1 \pmod{L}$ 암호화 $c = m^e \bmod n$, 복호화 $m = c^d \bmod n$ / 2048비트 이상 + 난수 패딩
엘가말 (이산대수)	임시 비밀값으로 둘만 아는 일회용 마스킹 키를 만들어 평문을 가림 매번 다른 암호문, 길이는 두 배, 임시값 재사용 금지
타원곡선 암호	점을 k번 더하기는 쉽고, P와 Q에서 k를 찾기는 어려움 같은 안전성에 훨씬 짧은 키 (RSA 2048 비트 수준 → 타원곡선 224비트)

핵심 정리 (2) : 키 교환

키 교환이란	공개키 암호 = 메시지를 암호화 / 키 교환 = 같은 대칭키를 만들어 냄 키 전송(한쪽이 만들어 전달) vs 키 합의(양쪽이 함께 생성)
디피-헬만	공개 : p, α / 비밀 : a, b → A = α^a, B = α^b 를 교환 양쪽 모두 α^(a×b) mod p 를 얻음, 안전성은 이산대수 문제
한계	상대를 확인하지 않음 → 중간자 공격 가능 전자서명 · 인증서로 상대를 확인하는 수단이 함께 필요

Thank You

E-mail: seyoung@kangwon.ac.kr